



FlatMate

Software Engineering I Projekt

Team

Denis

Test + CI/CD +
Entwicklung



Mykyta

Entwicklung +
Architektur



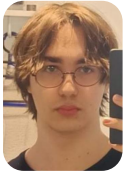
Kim

Design + Blog



Yaroslav

Entwicklung + Test



Leon

Projektmanagement +
Entwicklung



Unsere Vision



Problem

Chaotische Organisation in
WGs führt oft zu
unübersichtlichen Ausgaben,
fehlender Transparenz und
Frust, was zu vermeidbaren
Konflikten führt.

Zielgruppe

Unsere Anwendung richtet
sich an WGs, insbesondere an
Studierende und junge
Erwachsene, die ihren Alltag
digital und zentral organisieren
möchten.

MVP

Der MVP ermöglicht das
Anlegen einer WG, den Beitritt
von Mitgliedern, das Erfassen
gemeinsamer Ausgaben, das
erstellen einer Einkaufsliste
und ein automatisierter
Putzplan

FlatMate

Funktionale Anforderungen

Webapp

Die Anwendung soll über den Browser zugänglich und responsiv gestaltet sein (Desktop & Mobil)

Budgetverwaltung

Erfassung und Aufteilung gemeinsamer Ausgaben, automatische Schuldenberechnung

Putzplan

Erstellung, Zuweisung und Verwaltung von Reinigungsaufgaben für WG-Mitglieder

Einkaufsliste

Gemeinsame Liste zum Hinzufügen, Bearbeiten und Abhaken von Artikeln

Account erstellen / Login-System / WG-Account erstellen

Nutzer*innen können persönliche und WG-Accounts erstellen und verwalten, Möglichkeit, WG-spezifische Gruppen zu verwalten und Mitglieder hinzuzufügen

Nicht Funktionale Anforderungen

Benutzerfreundlichkeit

Die App soll eine intuitive, moderne Oberfläche haben, leicht navigierbar für neue Nutzer*innen.

Performance und Stabilität

Seiten sollen in wenigen Sekunden laden; die App bleibt auch bei hoher Nutzerzahl reaktionsschnell.

Wartbarkeit

Modularer Aufbau, um neue Features (z. B. Werbung, Challenges) einfach integrieren zu können.

Technologiestack

Language & Runtime

TypeScript - Node.js - npm

Frontend

React - Next.js App Router -
CSS

Backend

Next.js Route Handlers / API
Routes

Authentication & Security

NextAuth.js - bcryptjs

Database & Data Access

SQLite - Prisma Client

Validation & Quality

Zod - Vitest - ESLint

Deployment & Operations

Docker - Node Alpine - GitHub
Actions

Project Management

GitHub Projects

Warum dieser Technologiestack

Schnelle Entwicklung

Frontend, Backend und API-Logik liegen in einem gemeinsamen Projekt.

Wartbarkeit

Typisierung, Validierung und strukturierter Datenbankzugriff reduzieren Fehler.

MVP-Fokus

Leichtgewichtiges Setup ohne unnötige Infrastruktur.

Sicherheit

Geschützte Passwortspeicherung und serverseitige Prüfung sensibler Eingaben.

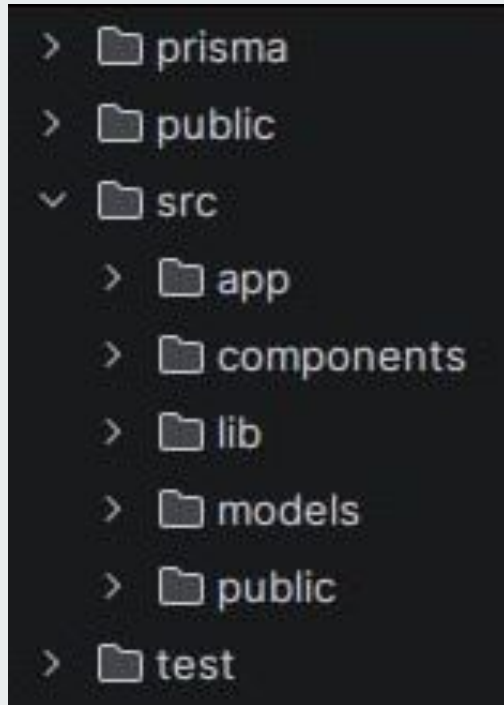
Qualitätssicherung

Automatische Prüfung von Code, Tests und Build in GitHub Actions.

Deployment Readiness

Docker prüft in der CI/CD-Pipeline, ob aus dem Build ein deploybares Image entsteht.

Modularität und Trennung der Verantwortlichkeiten



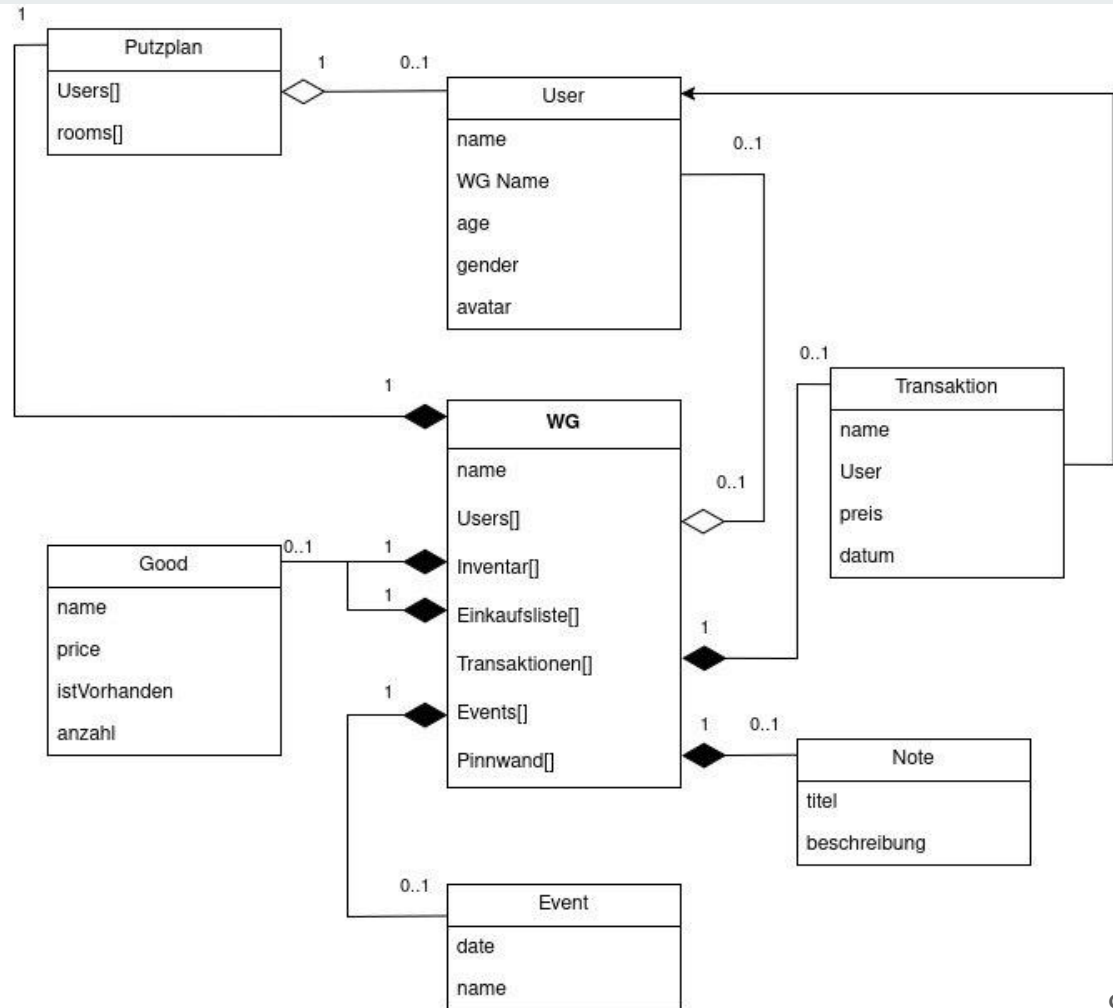
**modulare Struktur nach
Funktionsbereichen**

**leichtgewichtige und
projektgerechte Datenbanklösung**

**dokumentierte
Architekturentscheidungen als
gemeinsame Basis**

Architektur der Datenstruktur: Klassendiagramm

```
✓ models
  TS event.ts
  TS good.ts
  TS note.ts
  TS putzplan.ts
  TS transaction.ts
  TS user.ts
  TS wg.ts
```



Architektur der Datenstruktur: DB Tabellen

CleaningRating

CleaningRoom

CleaningWeekOverride

Event

Expense

ExpenseParticipant

Invite

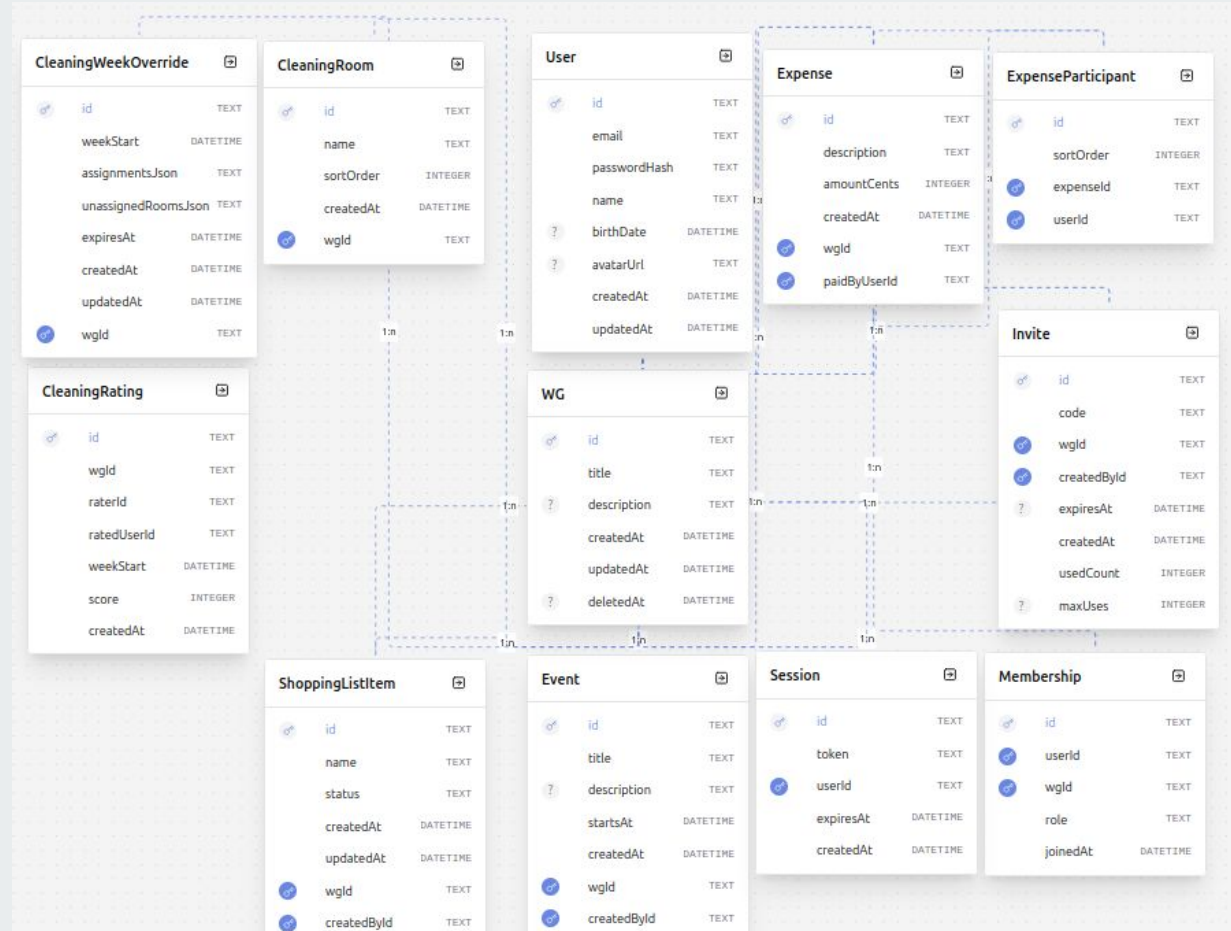
Membership

Session

ShoppingListItem

User

WG



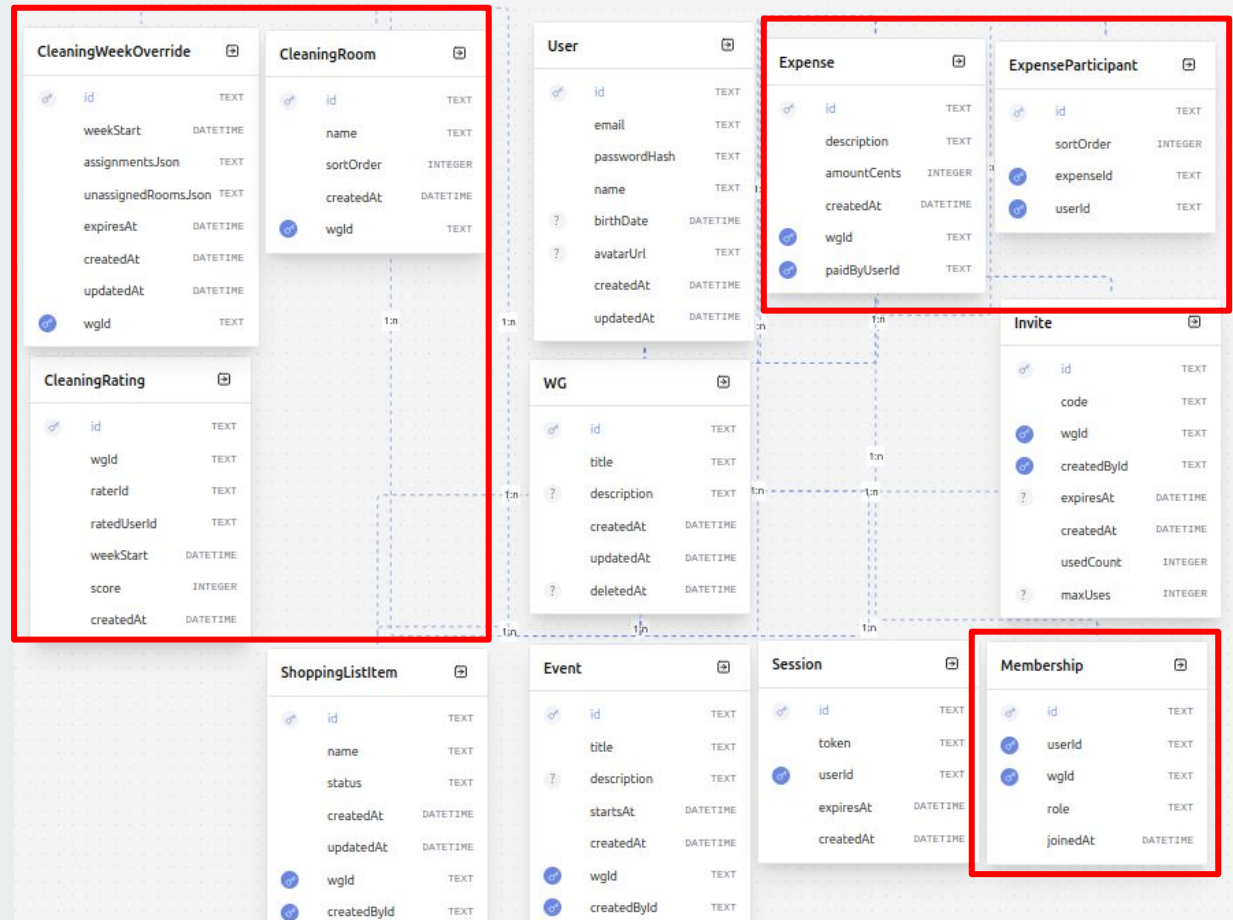
Tabellen- aufteilung nach 2. Normalform

Keine Vermischung

Eigene Tabellen

Weniger Redundanz

Klarere Struktur



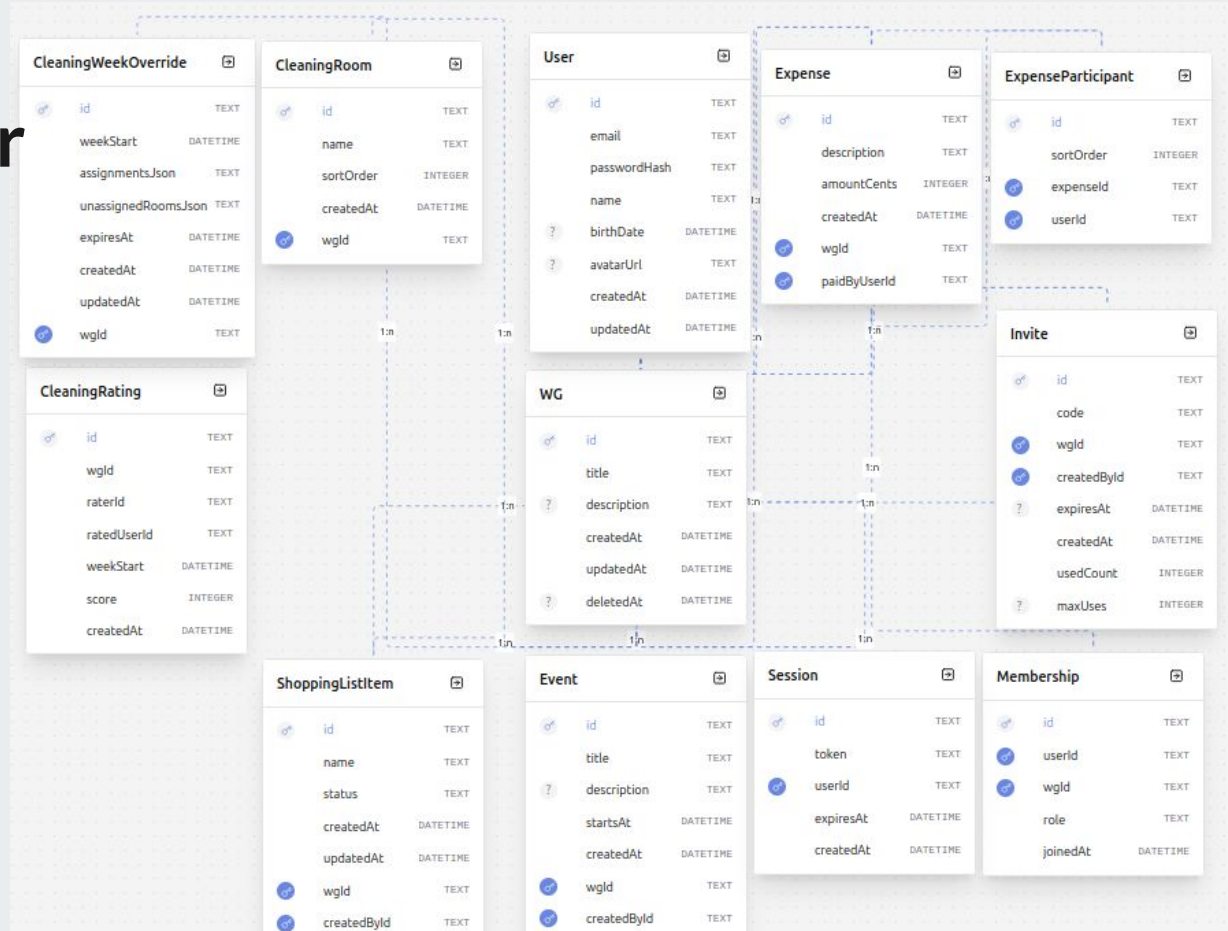
Was bringt uns diese Architektur praktisch

Konsistentere Daten

Leichtere Erweiterung neuer Features

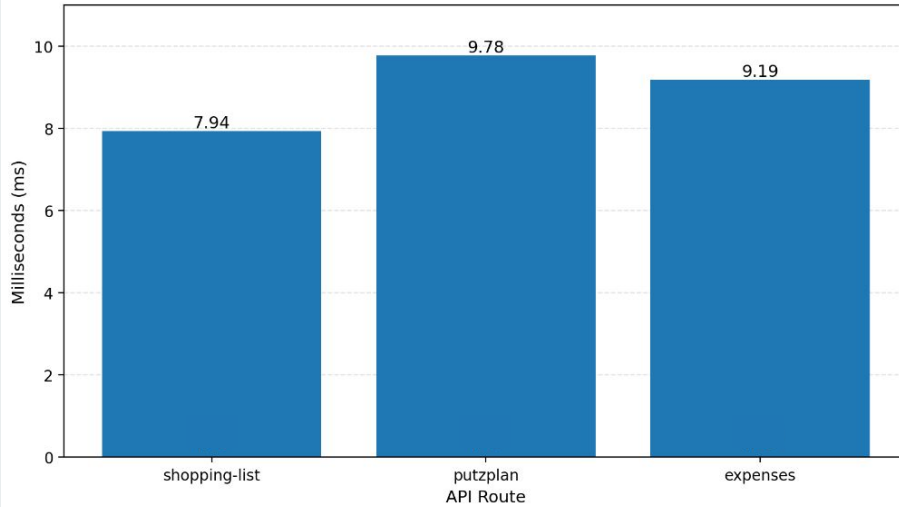
Klarere API- und Backend-Logik

Bessere Wartbarkeit im Team



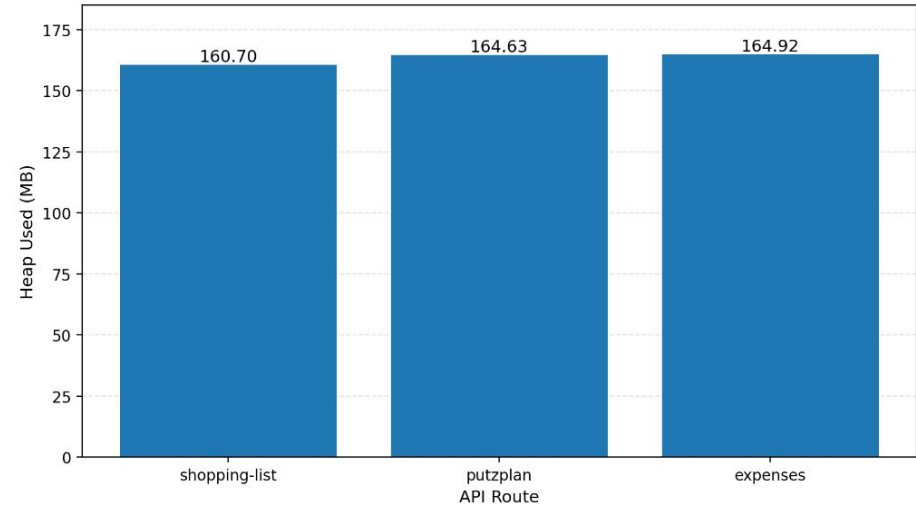
Durchschnittliche API-Antwortzeit

Average API Response Time



Alle drei Routen liegen im Durchschnitt unter 10 Millisekunden.

Durchschnittliche Speichernutzung



Die Werte liegen bei den getesteten Routen relativ nah beieinander

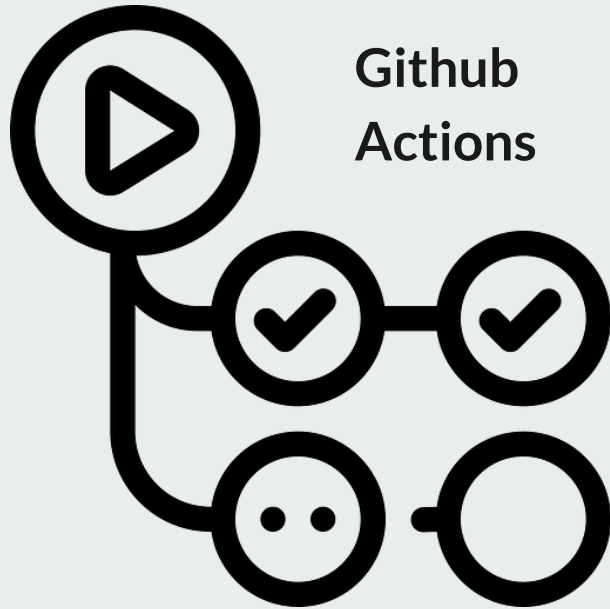
CI/CD Pipeline

Fehler früh erkennen

Die Qualität des Projekts sichern

Und manuelle Arbeit reduzieren

CI/CD Pipeline



Github
Actions

tests.yml

- Unit Tests
- Integration Tests
- Coverage Test



main.yml

- Build 
- Availability
- Response Time

Teststrategie

Unit-Tests für Validierungslogik, Hilfsfunktionen und Berechnungen

Integrationstests für Authentifizierung, Rechteprüfung,
Datenvalidierung und Datenbankzugriffe

Coverage-Ziel: 60 %

DEMO

<https://drive.google.com/file/d/1tQeJbqFokgUF-2aFGOKQWqpTK5RNEVWN/view?usp=sharing>

Demo Highlights

Putzplan

Automatisierter Putzplan, der trotzdem Anpassbar ist, trotzdem nicht viel Speicher benutzt (nur die angezeigten Wochen werden gespeichert)

Scoring System beim Putzplan

Alleinstellungsmerkmal unserer App und Motivation gut für die WG zu Putzen - coole Gamification

Mainpage

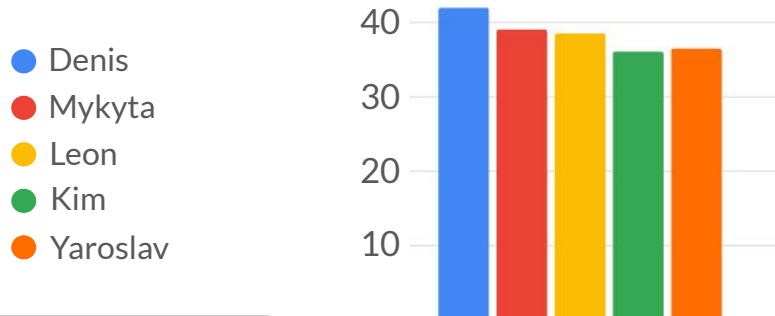
Super übersichtliche Mainpage, wo man alles wichtige auf einen Blick hat ohne überladen zu werden

Passwort Reset und Invitation Link

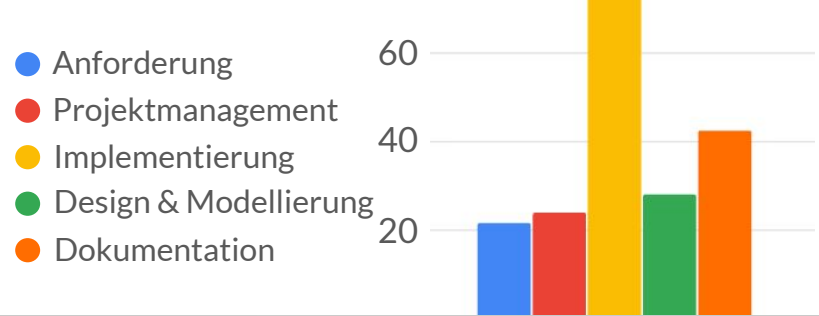
Einfache Handhabung möglicher schwieriger Themen - durch den Invite Link wurde z.B. das hinzufügen der Mitbewohner einfach umgesetzt und funktioniert für alle gut

Projektmanagement

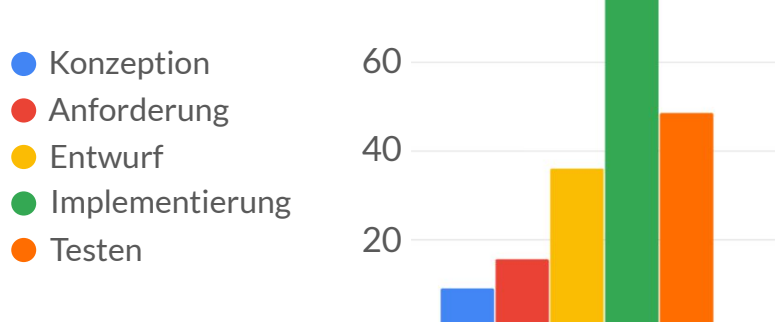
Stunden / Person



Stunden / Disziplin



Stunden / Phasen

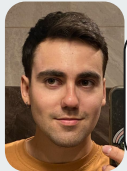


Interessante Fakten

ca. **190 Stunden** haben wir insgesamt bereits am Projekt **gearbeitet**

Eine ganze Arbeitswoche pro Kopf.

Hauptbeiträge



Denis

Backend & Auth: LogIn, Prisma-DB, Create-WG, Registrierung/Login. Dazu einen großteil der CI/CD-Infrastruktur



Kim

Doku & Blogs sowie Design (Figma, Mock-Ups) und Mitarbeit an Anforderungen/Diagrammen



Yaroslav

Feature-Implementierung (Putzplan-Page) plus Architektur/Modellierung, Testing und Qualität



Mykyta

Frontend (Navbar, Events, Dashboard, Register) plus viel UML- und Diagramm-Modellierung



Leon

Projektorga (Risk Management, Regeln) und Blogeinträge. Dazu Frontend-Tasks (Putzplan Scoring, Einkaufsliste, User Settings)

Lessons Learned

Schätzung ist schwer — vor allem wenn man mit neuen Tools arbeitet
→ mehr Puffer einplanen.

CI/CD früh aufsetzen — Fehler früh erkannt, weniger Handarbeit.

Konsequent Zeit tracken — erst dadurch saubere Auswertung möglich.

MVP-Fokus halten — leichtgewichtiges Setup, schneller lauffähig.

Spezialisierung schafft Insel-Wissen — nächstes Mal mehr Pair-Work.

Unsere(r) Highlights / Stolz

Die Aufgabenverteilung im Team - jeder hat +- gleich viel beigetragen

Design + gute Umsetzung der Theorie

Branches bei GitHub + CI/CD bei GitHub Actions

Aus einem guten fachlichen Konzept eine real umsetzbare und stabile Anwendung entsteht

Prisma ORM + Prisma Studio



Thank you

FlatMate